Welcome to My SYDE671 Portfolio!

Part One: Fun with Filters

1.1 Convolution from Scratch

Images are represented by matrices of pixel values, and image filters can be represented as smaller matrices. To apply the filter, we can slide the filter kernel over every pixel, multiply the weights, and sum the outcome into one output pixel. This is called convolution, and it is a fundamental operation in image processing and computer vision.

Here, I've started with building my own convolution function in Python. The four-loop version goes over every image pixel and every kernel entry. The two-loop version goes over the image pixels only, and uses NumPy to do the multiplication and summing over each kernel-sized window. They give the same output, but the second one is faster. I've attached the code and the outputs of these operations below.

Boundaries: both versions use zero padding, so the output is the same size as the input. The kernel is flipped in both directions, so we get a convolution rather than a correlation. For an even kernel size the extra padded row/column goes after the image, which is exactly what `scipy.signal.convolve2d(..., mode="same", boundary="fill", fillvalue=0)` does, so the outputs can be compared directly. The consequence of zero padding is that pixels near the border are blended with black, which darkens the border of a blurred image and creates a fake edge along the border for the derivative filters (I deal with that in Part 1.2).

▼ **Show convolution implementation**

```
def pad_zeros(image, kshape):
    kh, kw = kshape
    return np.pad(image, ((kh // 2, kh - 1 - kh // 2), (kw // 2, kw - 1 - kw // 2)))  # zeros by default

def convolve2d_four_loops(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Convolution with four explicit for loops (image rows/cols, kernel rows/cols)."""
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    h, w = image.shape
    kh, kw = kernel.shape

    padded = pad_zeros(image, kernel.shape)
    flipped = kernel[::-1, ::-1]  # flip both axes: true convolution

    output = np.zeros((h, w), dtype=np.float64)
    for r in range(h):
        for c in range(w):
            total = 0.0
            for i in range(kh):
                for j in range(kw):
                    total += padded[r + i, c + j] * flipped[i, j]
            output[r, c] = total
    return output

def convolve2d_two_loops(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Convolution with two explicit for loops; the kernel sum is done with NumPy."""
    image = np.asarray(image, dtype=np.float64)
```

```
    kernel = np.asarray(kernel, dtype=np.float64)
    h, w = image.shape
    kh, kw = kernel.shape

    padded = pad_zeros(image, kernel.shape)
    flipped = kernel[::-1, ::-1]

    output = np.zeros((h, w), dtype=np.float64)
    for r in range(h):
        for c in range(w):
            output[r, c] = np.sum(padded[r:r + kh, c:c + kw] * flipped)
    return output
```

The filters used are a 9×9 box filter and the finite difference operators. The comparison against SciPy times each implementation and reports the largest absolute difference from `scipy.signal.convolve2d`:

▼ **Show convolution benchmarking code**

```
BOX = np.ones((9, 9)) / 81.0              # 9x9 box (averaging) filter
DX = np.array([[-1, 1]], dtype=float)     # finite difference in x
DY = np.array([[-1], [1]], dtype=float)   # finite difference in y

gray = cv2.imread("Janet.jpg", cv2.IMREAD_GRAYSCALE).astype(float)

for name, k in {"Box 9x9": BOX, "Dx": DX, "Dy": DY}.items():
    out4, t4 = timed(convolve2d_four_loops, test, k)
    out2, t2 = timed(convolve2d_two_loops, test, k)
    ref, ts = timed(convolve2d, test, k, "same", "fill", 0)      # SciPy, zero padded
    print(name, t4, t2, ts, np.abs(out4 - ref).max(), np.abs(out2 - ref).max())
```

Runtime (and agreement with SciPy) on a 64×64 crop of the image, since the four-loop version is too slow in pure Python to run on the whole picture:

Filter	Four loops (s)	Two loops (s)	SciPy (s)	max |four loops − SciPy|	max |two loops − SciPy|
Box 9×9	0.185	0.020	0.0009	1.7e-13	7.1e-14
D_x	0.006	0.018	0.0001	0	0
D_y	0.006	0.034	0.0002	0	0

All three methods produce the same results, error being around 1e-13 or exactly zero. For the 9×9 box filter, the two-loop version is roughly an order of magnitude faster than four loops because NumPy does the 81 multiplications in one vectorized call, and SciPy is faster again. For the tiny 1×2 and 2×1 difference filters the kernel loops are so short that the four-loop version is actually quicker: calling `np.sum` on a two-element window costs more than just looping. The vectorization only pays off when the kernel has many entries.

Image of myself convolved with a 9x9 blurring filter using four loops.

Image of myself convolved with a 9x9 blurring filter using two loops.

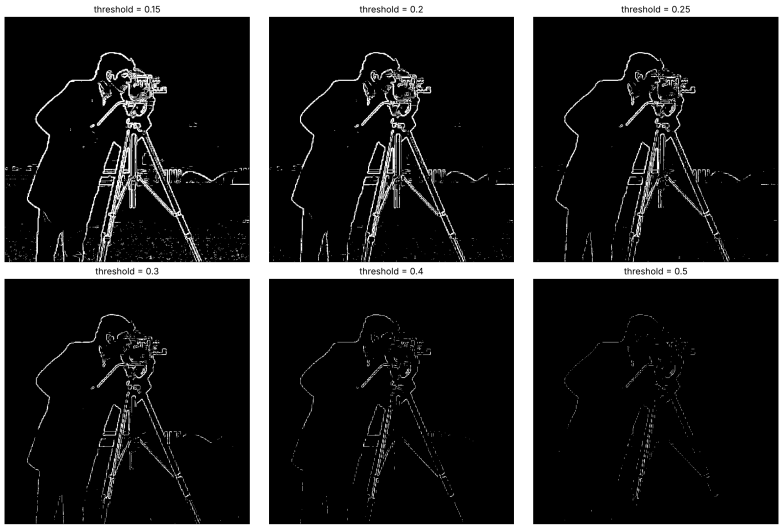
1.2 Finite Difference Operators

Mathematically, a derivative will calculate how fast something is changing, so an image derivative will highlight regions of rapid intensity change, which are edges. In two dimensions, we can take derivatives in x and in y, giving us Ix and Iy. However, we know conceptually that edges are not necessarily exclusively present in a single direction, so there is value in acquiring the gradient magnitude, which will tell us how strong edges are regardless of direction.

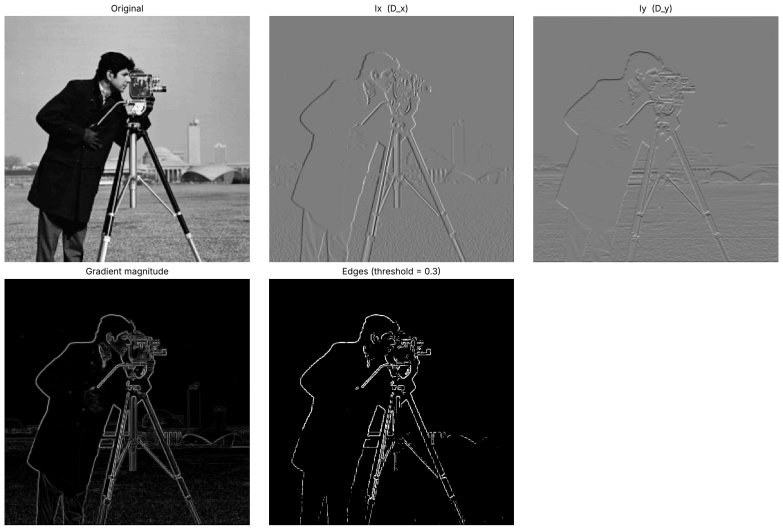
▼ Show finite-difference implementation

```
DX = np.array([[1, 0, -1]], dtype=float)
DY = np.array([[1], [0], [-1]], dtype=float)

ix = convolve2d(img, DX, mode="same", boundary="fill", fillvalue=0)
iy = convolve2d(img, DY, mode="same", boundary="fill", fillvalue=0)
mag = np.hypot(ix, iy)                # gradient magnitude, sqrt(Ix^2 + Iy^2)

# blank the zero padding
mag[[0, -1], :] = 0
mag[:, [0, -1]] = 0
edges = mag > 0.3                     # binarize with a threshold
```

We can have a python script calculate these gradients, then threshold them so that a magnitude of a certain strength can be classified as an edge. This threshold is going to be image dependent: a threshold too low will hold noise, while too high will delete the true edges. Using this cameraman image, I have calculated the gradients and applied different thresholds to see how the edges are affected. The results are shown below. The left image is showing the gradient magnitude map of the cameraman image thresholded at various levels. The right image is me having selected 0.3 as the best thresholding level, and showing the original image, the x and y derivatives, the gradient magnitude, and the thresholded edges.



Different thresholds of cameraman after gradient calculation.



Original cameraman image with derivatives, gradient magnitude, and 0.3 threshold.

0.3 is what I selected to be the "best" threshold. This is a trade-off, and I chose to lean toward removing noise. At the lowest threshold (0.15) the grass in the foreground turns into a speckle of false edges, and the horizon clutter is noisy. At 0.3, the strong edges (the coat, head, camera and tripod legs) are clean and continuous, and the grass speckle is gone. The price is that faint, low-contrast edges, such as the distant buildings and the horizon line, mostly drop out. Raising the threshold further (0.4–0.5) starts to break up even the strong edges, such as the tripod legs, so 0.3 was the best compromise. The border pixels are also blanked, since zero padding produces a bright artificial edge around the image.

1.3 Derivative of Gaussian (DoG) Filter

If we look closely at the 0.3-threshold image, you might have noticed that there is still some noise that is captured in the magnitude gradient thresholding. A way to tackle this problem is to smooth the image before calculating the gradients. A common way to do this is to use a Gaussian filter.

The Gaussian filter is built with `cv2.getGaussianKernel` to make a 1D Gaussian, then taking an outer product of it with itself to get the 2D kernel. I used σ = 2 with a 13×13 kernel (about 6σ wide, so the tails are negligible).

<details>
<summary>▼ Show DoG implementation</summary>

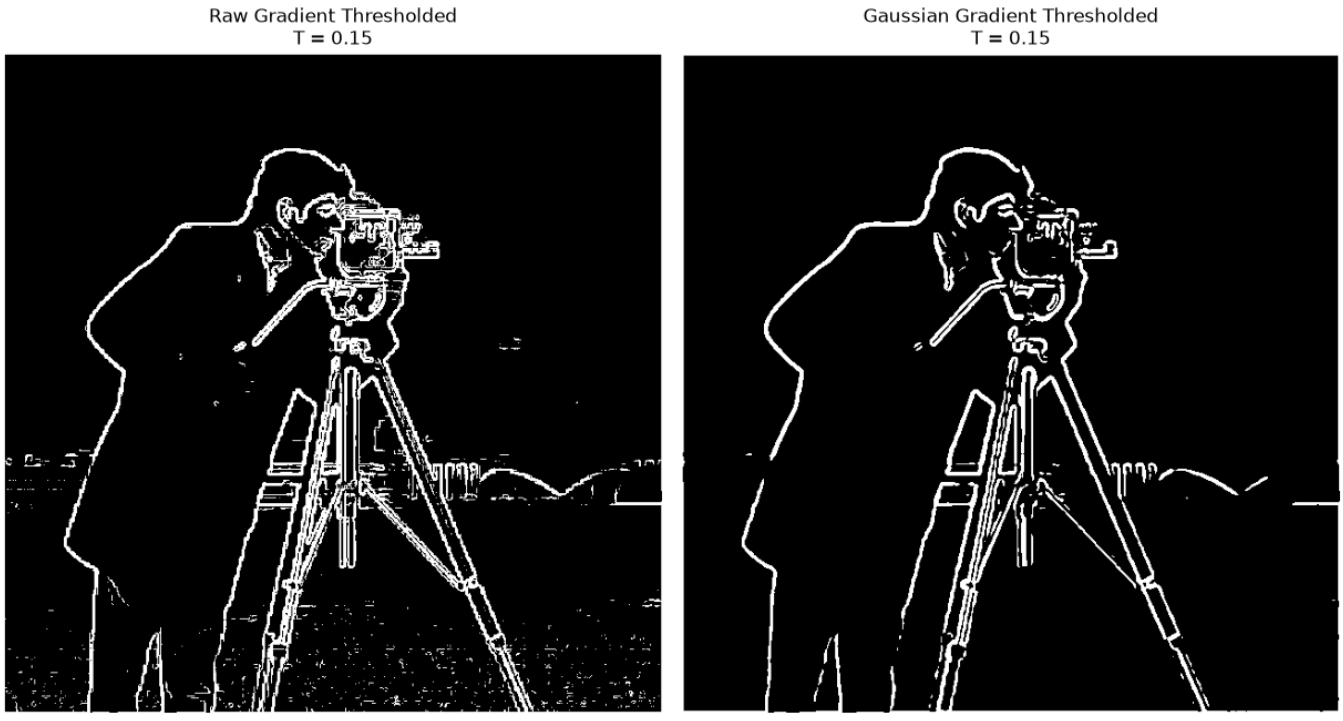
```
def gaussian_2d(ksize, sigma):
    g = cv2.getGaussianKernel(ksize, sigma).ravel()   # 1D Gaussian
    return np.outer(g, g)                             # 2D Gaussian (sums to 1)

G = gaussian_2d(13, 2.0)

# blur the image, then differentiate the blurred image
blurred = conv(img, G)
bl_x, bl_y = conv(blurred, DX), conv(blurred, DY)

# build the DoG filters by convolving G with D_x and D_y ("full" keeps every tap)
DoG_x = convolve2d(G, DX, mode="full")
DoG_y = convolve2d(G, DY, mode="full")
dog_x, dog_y = conv(img, DoG_x), conv(img, DoG_y)
```

</details>

In this case, we used a Gaussian filter compared to the box filter from Part 1.1. It's kind of hard to compare a photo of me to the cameraman, so I've displayed the original image, the box filter, and the Gaussian filter below. We can see that the box filter provides a more uniform blur while the Gaussian does a better job at preserving the original features. This is because the Gaussian filter takes a weighted sum of the pixels nearby based on their distance from the pixel in question, so edges (which will likely span multiple pixels) will be preserved better than noise, which might only affect a single pixel at a time.

Box and Gaussian filter comparison.

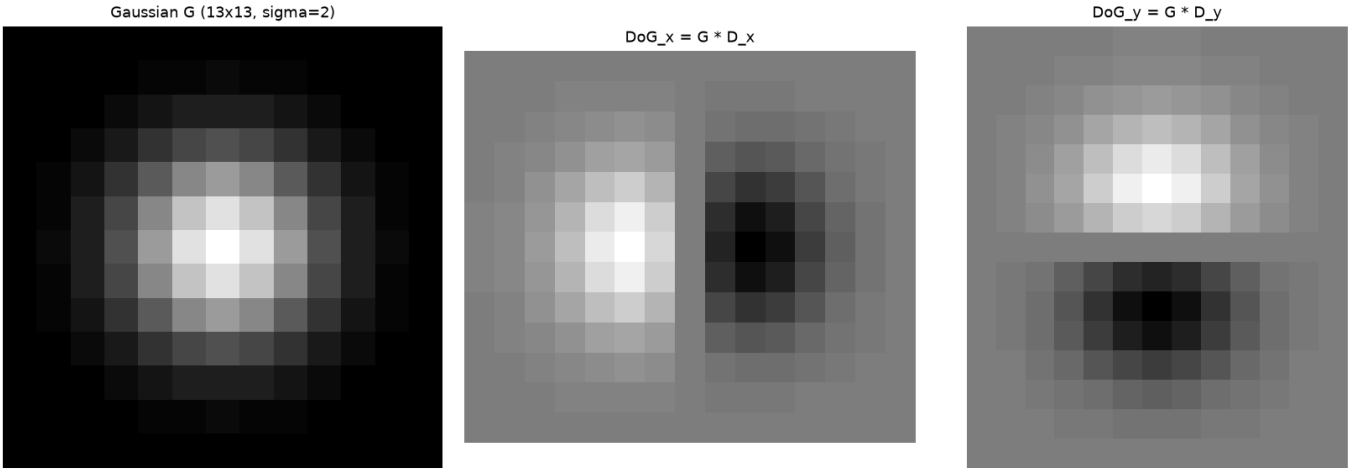
Gradient map not using and using the Gaussian filter



Gradient thresholding not using and using the Gaussian filter

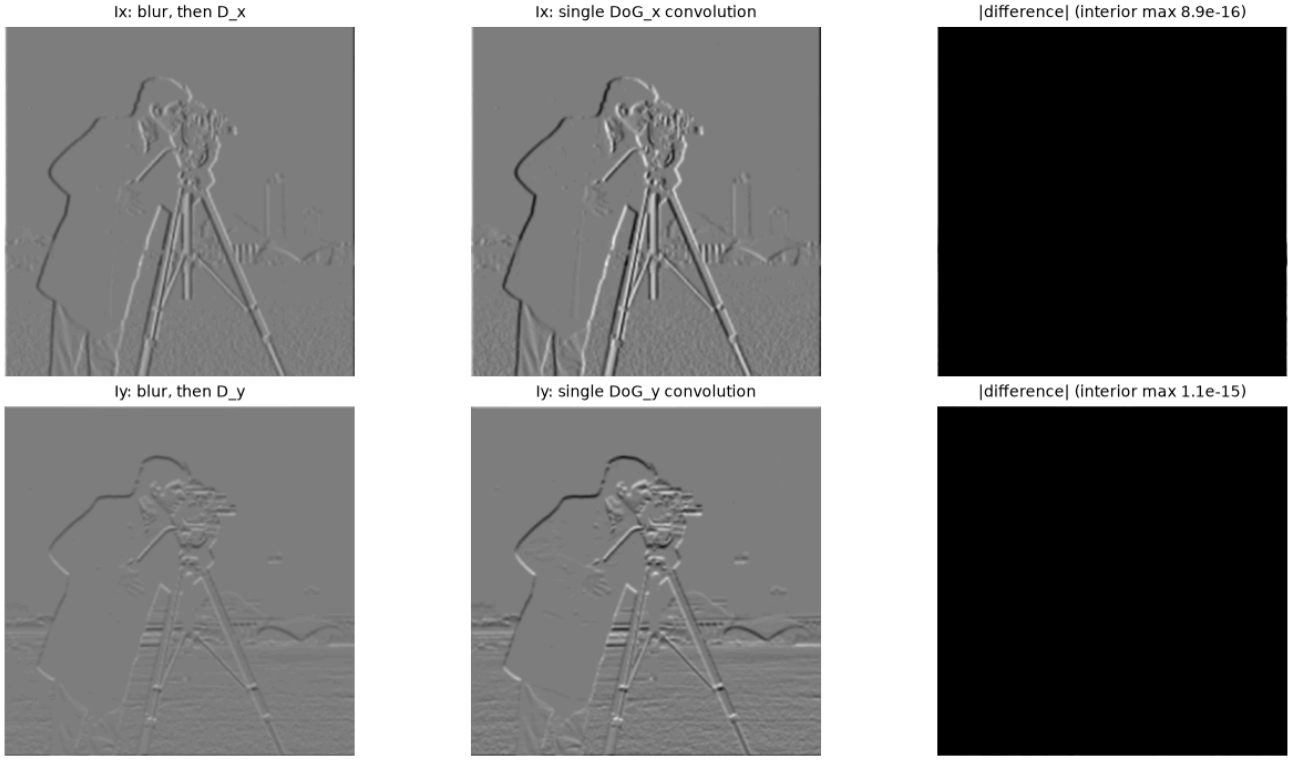
We can see that because the Gaussian helps us reduce some of the noise at an earlier stage, we can use a lower threshold from the gradient map (about 0.12 here), while before we had to use a higher one (0.25–0.3) to avoid capturing noise. The smoothed gradients have a smaller overall magnitude, which is why the absolute threshold is lower, but the noise is suppressed much more than the real edges. Now, we can see the horizon line in the cameraman image clearly compared to before.

Turns out we can try to simplify the process by combining the Gaussian filtering and the gradient calculation into a single convolution. Convolutions are associative, so convolving the image with a Gaussian filter and then convolving the result with a derivative filter is equivalent to convolving the image with a single filter that is the convolution of the Gaussian and derivative filters. This is called a Derivative of Gaussian (DoG) filter, and it can be used to calculate gradients while also smoothing the image. Here are the Gaussian and the two DoG filters displayed as images:



The Gaussian G (13x13, $\sigma = 2$) and the derivative of Gaussian filters $DoG_x = G * D_x$ and $DoG_y = G * D_y$. DoG_x has a bright lobe on one side and a dark lobe on the other: it smooths and takes a horizontal difference at once.

We'll make sure the single-convolution version matches the two-step process! The results are shown below.



DoG filter comparison to two-step process

The largest absolute difference between "blur then differentiate" and the single DoG convolution is about 9e-16 for I_x and 1e-15 for I_y away from the border, which is floating-point rounding error, so the two are the same result. Over the whole image the maximum difference is larger (about 0.3) but it comes entirely from the border: with zero padding, the

two-step process pads the image once for the blur and then again for the derivative, while the DoG filter pads only once, so the first few pixels at the edge differ. Away from the border (more than half a kernel width in) they match exactly.

Bells & Whistles

Another way to visualize the image gradients is to add a colour map. For example, if we use HSV (Hue, Saturation, Value) space, we can map the gradient magnitude to the value channel, the gradient direction to the hue channel, and keep the saturation constant. Now at a glance, we can see in what "directions" the edges are oriented. For example, the edge between the background and the cameraman's back is green, meaning that the gradient is oriented in the direction from his back to the background, and in the original image, this corresponds to the edge of the cameraman's back being darker and the edge of the background being lighter.

HSV gradient visualization

To do this without using anything fancy, such as using a built-in angle function, we use the "diamond angle", which only needs arithmetic vs needing `arctan2`. With $p = I_x / (|I_x| + |I_y|)$, which lies in $[-1, 1]$ and increases monotonically with the true angle, the hue is $(1 - p)/4$ when $I_y \geq 0$ (angles 0°–180°, hue 0 to 0.5) and $(3 + p)/4$ when $I_y < 0$ (angles 180°–360°, hue 0.5 to 1). It lands on the right values at 0, 45, 90, ... degrees and is off by only a few degrees in between, which doesn't matter for colouring. Saturation is 1 and value is the gradient magnitude (normalized by its 99.5th percentile so a few extreme pixels don't wash out the picture). I computed the gradient from the DoG-filtered image so the orientation isn't noisy. One detail is the sign: row numbers grow downward in an image but the colour wheel has y pointing up, so I negate I_y before computing the angle. The colour wheel at the right of the figure below is the legend for the mapping (right-pointing gradient = red; left = cyan).

▼ **Show gradient-orientation implementation**

```
def pseudo_angle(x, y):
    """Direction of the vector (x, y) as a number in [0, 1)."""
    l1 = np.abs(x) + np.abs(y)
    p = np.divide(x, l1, out=np.zeros_like(x, dtype=float), where=l1 > 0)   # x / (|x| + |y|)
    return np.where(y >= 0, 1 - p, 3 + p) / 4

def orientation_rgb(ix, iy, mag):
    hue = pseudo_angle(ix, iy)
    value = np.clip(mag / np.percentile(mag, 99.5), 0, 1)
```

```
    return hsv_to_rgb(np.dstack([hue, np.ones_like(hue), value]))

rgb = orientation_rgb(dog_x, -dog_y, dog_mag)
```

Part Two: Fun with Frequencies

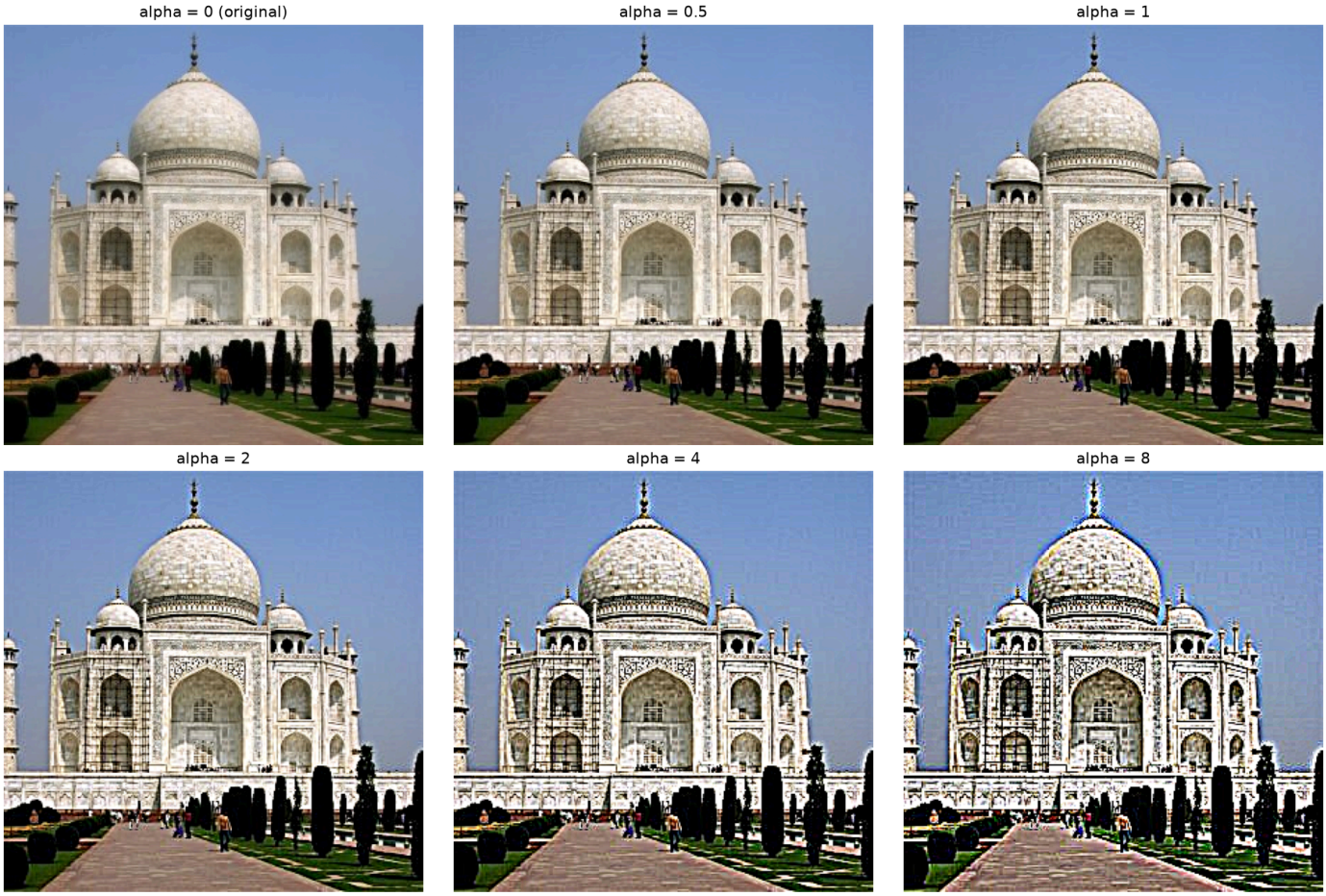
2.1 Image "Sharpening"

Since we know that edges tend to be higher frequency components in the image, we can artificially "sharpen" an image by isolating the high frequency components and adding them back to the original image. This is called unsharp masking. To be able to program this in an intuitive way, the sharpening effect can be expressed by this equation: **sharp = I + α * (I - I * G)**. G is the Gaussian, and we can control its effect on the image by changing the **σ**-value. **α** is a scaling factor we can use to adjust the strength of the sharpening effect. The Gaussian is a low-pass filter, so **I * G** keeps only the low frequencies, and **I - I * G** is what the blur removed: the high frequencies. I've shown the Taj Mahal image below with different **σ** and **α** values to see how the sharpening effect is affected.

Because convolution is linear, the whole operation can be combined into a **single convolution**: with δ as the impulse (identity) filter, **sharp = I * ((1 + α)δ − αG)**, the "unsharp mask filter". It is just a big positive spike at the centre with a negative Gaussian skirt around it.

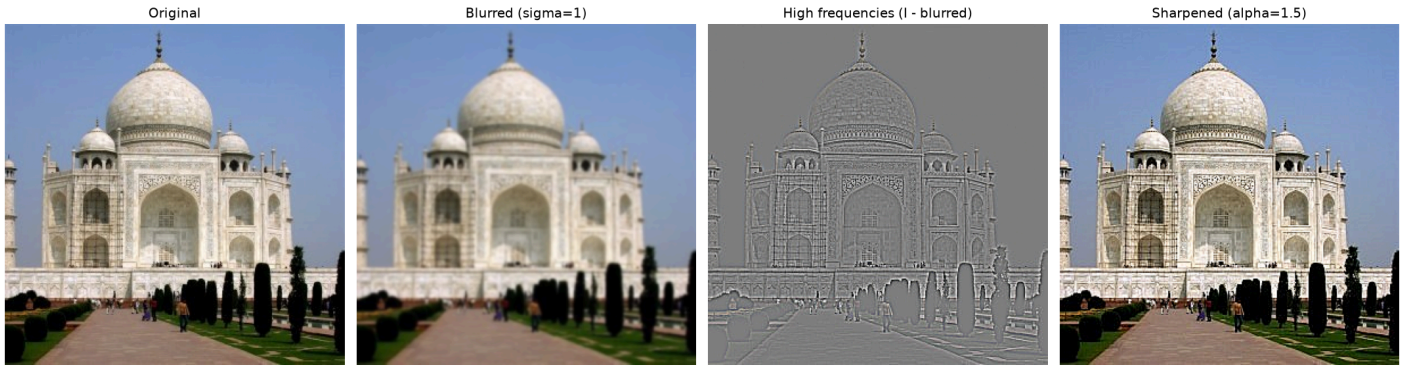
▼ Show unsharp masking implementation

```
def gaussian_kernel(sigma):
    ksize = 2 * int(np.ceil(3 * sigma)) + 1          # covers +/- 3 sigma
    g = cv2.getGaussianKernel(ksize, sigma)
    return g @ g.T                                    # 2D = outer product

def unsharp_kernel(sigma, alpha):
    """(1 + alpha) * delta - alpha * G: the whole filter as a single kernel."""
    G = gaussian_kernel(sigma)
    delta = np.zeros_like(G)
    delta[G.shape[0] // 2, G.shape[1] // 2] = 1
    return (1 + alpha) * delta - alpha * G

blurred = convolve(img, gaussian_kernel(sigma))           # low frequencies
detail  = img - blurred                                   # high frequencies
sharp   = convolve(img, unsharp_kernel(sigma, alpha))     # single convolution = img + alpha * detail
```

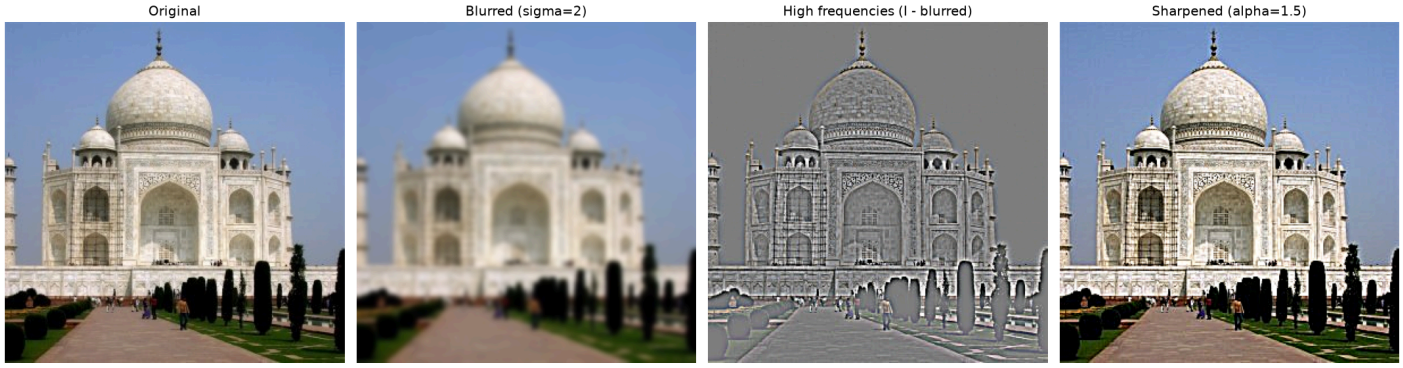


Alpha values comparison, sigma = 2.0

In the Taj Mahal image, the edges of interest would be the marble inlay, the arch details, scaffolding and tree outlines. These are removed with the Gaussian blur, so we're seeing if they're added back with the unsharp mask. We can see that **α = 0** to **α = 1** gives us a subtle sharpening effect, while higher values of **α** result in a haloing effect, and even higher values can lead to over-sharpening and noise amplification. I selected **α = 1.5** to be a good value to proceed, as we can see the finer details such as the dome texture and scaffolding are recovered without introducing too much noise or haloing. Now, we can see how the **σ** value affects the sharpening effect. In each of the "process" figures below, the four panels are, from left to right: the original, the blurred image (**I * G**), the high frequencies (**I − I * G**, shown around mid-gray), and the sharpened result.

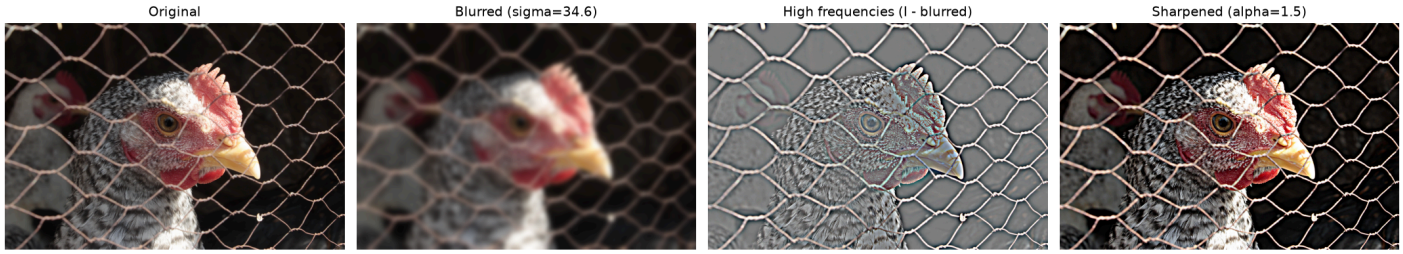


Gaussian blur then unsharp masking, **α = 1.5**, **sigma = 1.0**

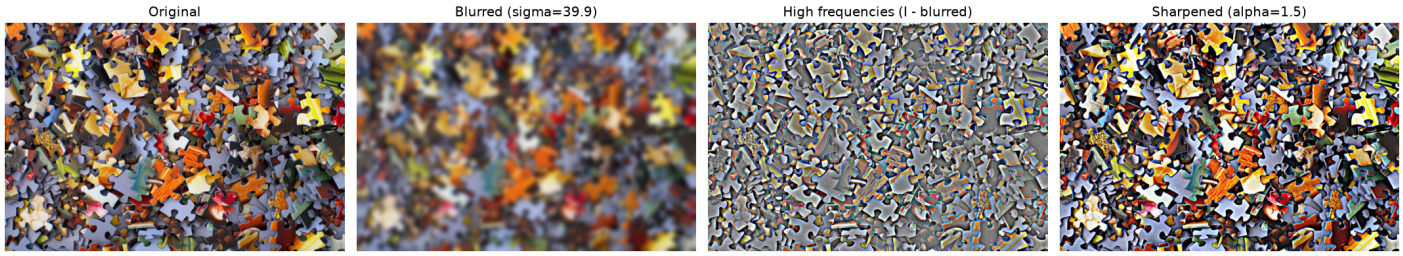


Gaussian blur then unsharp masking, **α = 1.5**, **sigma = 2.0**

We can see that **σ = 2.0** more severely blurs the original image, so the unsharp masking does a worse job at recovering the lost information because more of the high-frequency details are lost in the blurring process. I tried this Gaussian blur then unsharp mask on two more images, the chicken and the puzzle pieces! Since the images were higher resolution, I needed to use a greater sigma value. The script auto-sets sigma to the longest side of the image in pixels divided by 150.



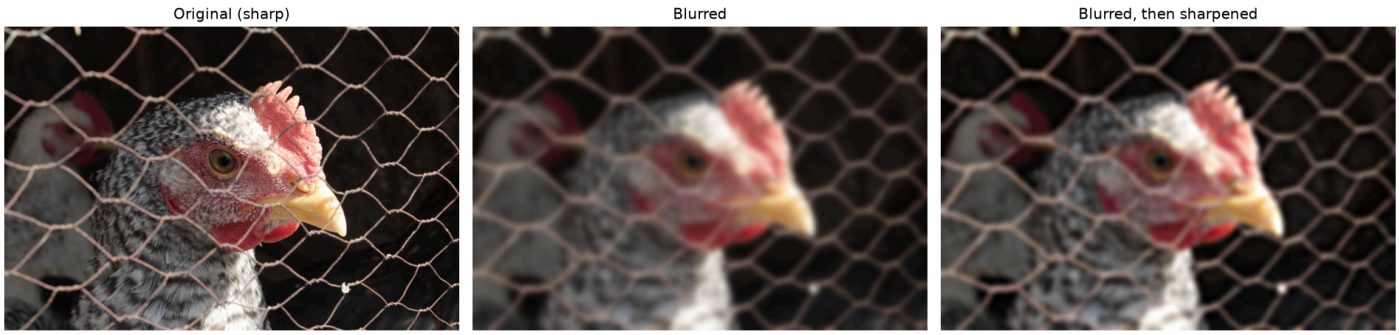
Gaussian blur and unsharp masking with a chicken



Gaussian blur and unsharp masking with a puzzle

Now comes the moment of truth: evaluating whether a blurred image can be recovered through sharpening. To see how much the unsharp mask can really recover, I started from sharp images (the Taj Mahal and the chicken), blurred them with the Gaussian, and then applied the unsharp mask (α = 1.5) to the blurred version. In each figure the panels are the original sharp image, the blurred image, and the blurred-then-sharpened image.

Taj Mahal: original, blurred (σ = 2), and blurred then sharpened (α = 1.5)

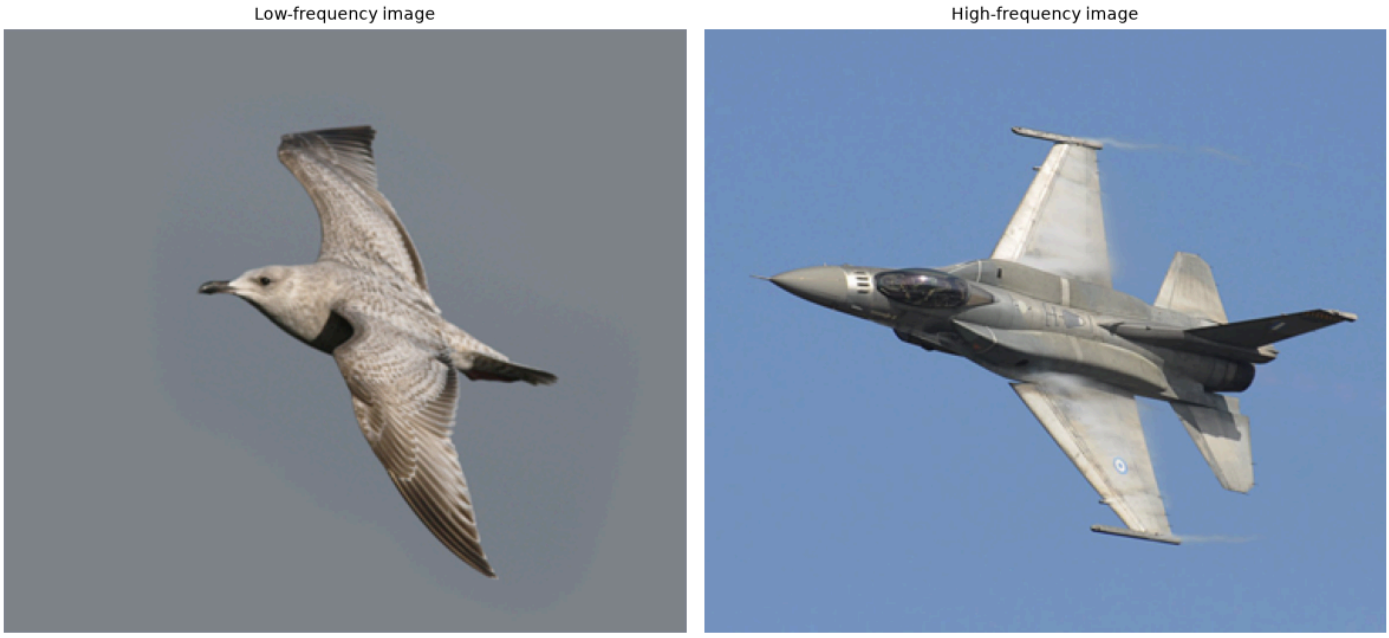


Chicken: original, blurred, and blurred then sharpened (α = 1.5)

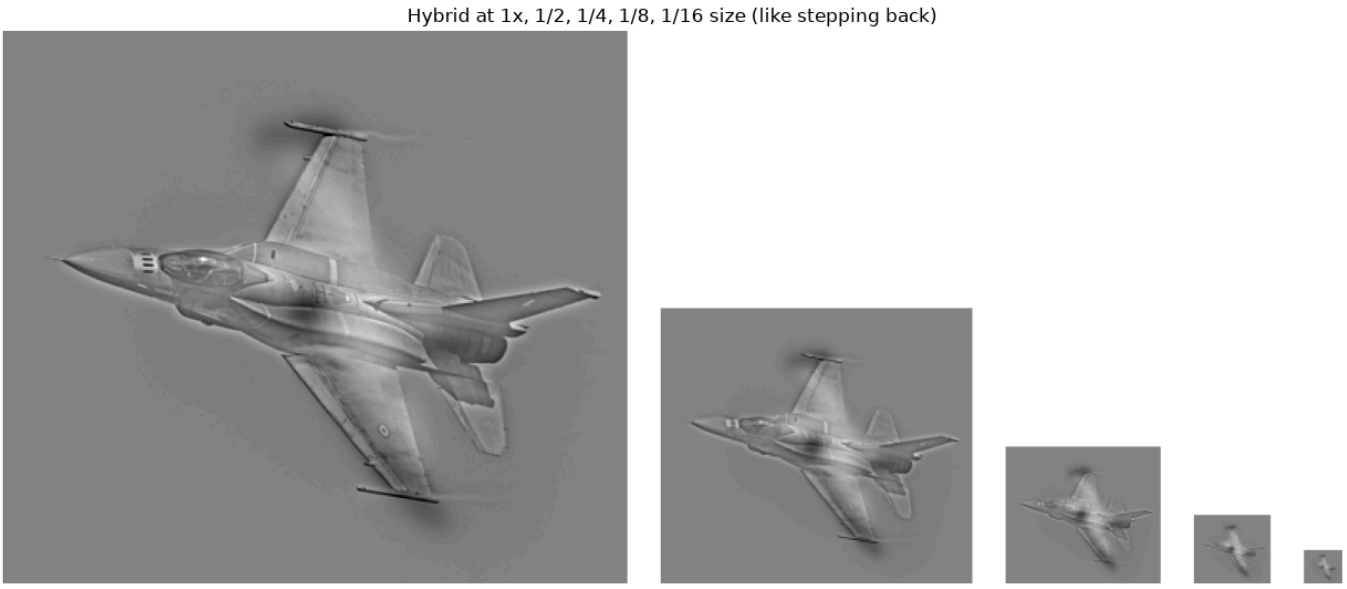
Measured against the original, the mean squared error (pixel values in 0–1) drops from 0.0046 for the blurred Taj Mahal to 0.0032 after sharpening, and from 0.0090 to 0.0067 for the chicken, so sharpening gets back roughly a quarter to a third of the error. Visually, the strong edges look crisper again (the dome and arch outlines, the trees), but the result is not the original: in the Taj Mahal the scaffolding and the fine marble inlay stay soft. Unsharp masking can only boost frequencies that survived the blur; details that the Gaussian wiped out completely (the finest texture) cannot be recovered, and the sharpened image shows slight halos along strong edges and amplified noise instead. It makes the image *look* sharper rather than restoring information.

2.2 Hybrid Images

Creating hybrid images tests an interesting concept in which high frequencies dominate our perception only when we're close enough to see them. Otherwise, from afar, we only really see the low-frequency components. So, if we combine the high frequencies of one image with the low frequencies of another, we can create an image that looks different depending on the viewing distance.



Images of a bird and plane



Hybrid image of a bird and plane at different distances, σ_{low} = 8.0 σ_{high} = 1.8

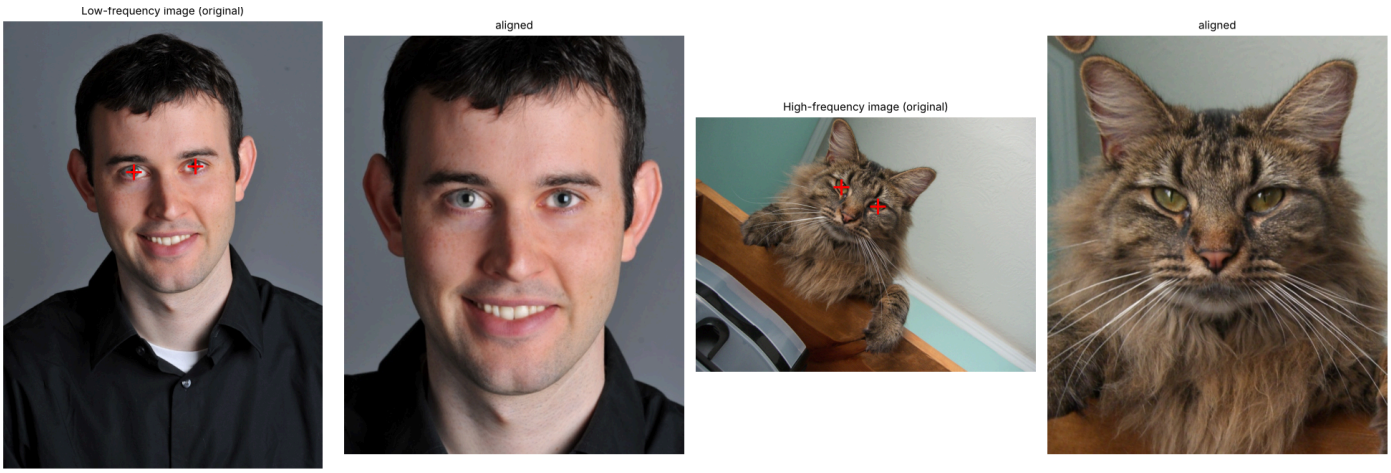
How we accomplish this is to low-pass filter one image and high-pass filter the other image. Then we average or sum the two images (I average it). The low-pass is a Gaussian-blurred version of the first image, and the high-pass is the second image minus its own Gaussian-blurred version (the impulse filter minus the Gaussian). The two images first have to be aligned (I pick the two eyes in each image, then rotate, scale and translate each image so those two points land on the same spot), since misaligned features break the illusion.

▼ Show hybrid-image implementation

```
def align(img, pts):
    """Rotate/scale/translate so the two points land on fixed horizontal positions."""
    cx, cy = REF_CENTER
    target = np.float32([[cx - REF_DIST / 2, cy], [cx + REF_DIST / 2, cy]])
```

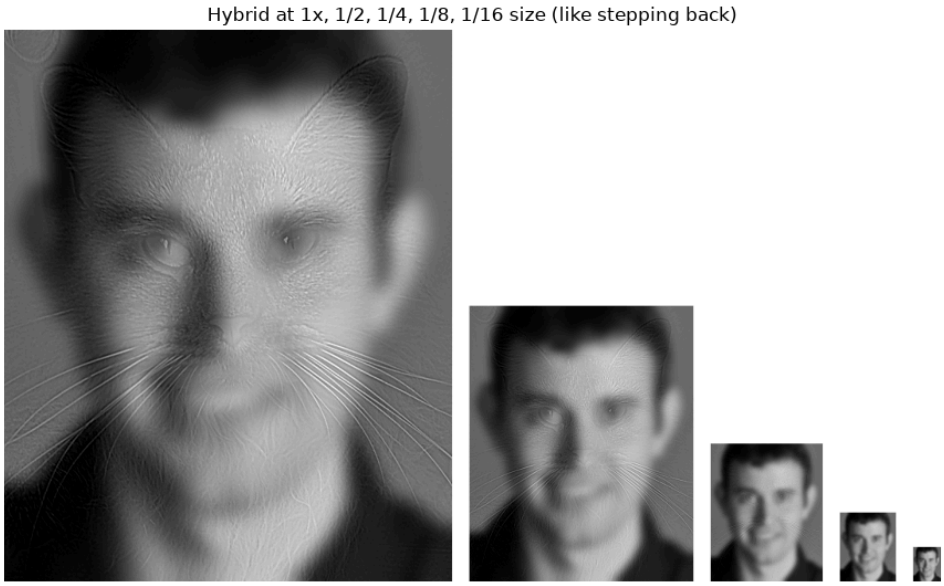
```
    M, _ = cv2.estimateAffinePartial2D(pts, target, method=cv2.LMEDS)
    return cv2.warpAffine(img, M, (CANVAS_W, CANVAS_H), flags=cv2.INTER_LINEAR,
                          borderMode=cv2.BORDER_REFLECT)

def hybrid(low_img, high_img, s_low, s_high):
    low = blur(low_img, s_low)                    # low-pass:  G_low * image_a
    high = high_img - blur(high_img, s_high)      # high-pass: image_b - G_high * image_b
    return low, high, np.clip(low + high, 0, 1)

def log_spectrum(g):                              # what is plotted in the frequency figures
    return np.log(np.abs(np.fft.fftshift(np.fft.fft2(g))) + 1e-8)
```

In terms of the Gaussian filters we will be using for the low-pass and high-pass filtering, we will have to make some choices in σ. For Derek and Nutmeg, a σ_{low} (the σ for the low-pass filter) of 7.0 keeps details coarser, meaning that his features are preserved, but not his skin texture. A σ_{high} (the σ for the high-pass filter) of 3.5 keeps the details finer, which are the fur and whiskers. A larger σ_{low} removes more of Derek's detail, so he is only recognizable once we step far back or shrink the image, and a smaller σ_{high} restricts the cat to only the very finest detail, which disappears at a shorter distance, so the hybrid turns into Derek sooner.



Aligned images of Derek and Nutmeg

Hybrid image of Derek and Nutmeg at different distances, σ_{low} = 7.0, σ_{high} = 3.5



Hybrid image of Derek and Nutmeg at different distances, σ_{low} = 10.0, σ_{high} = 1.5

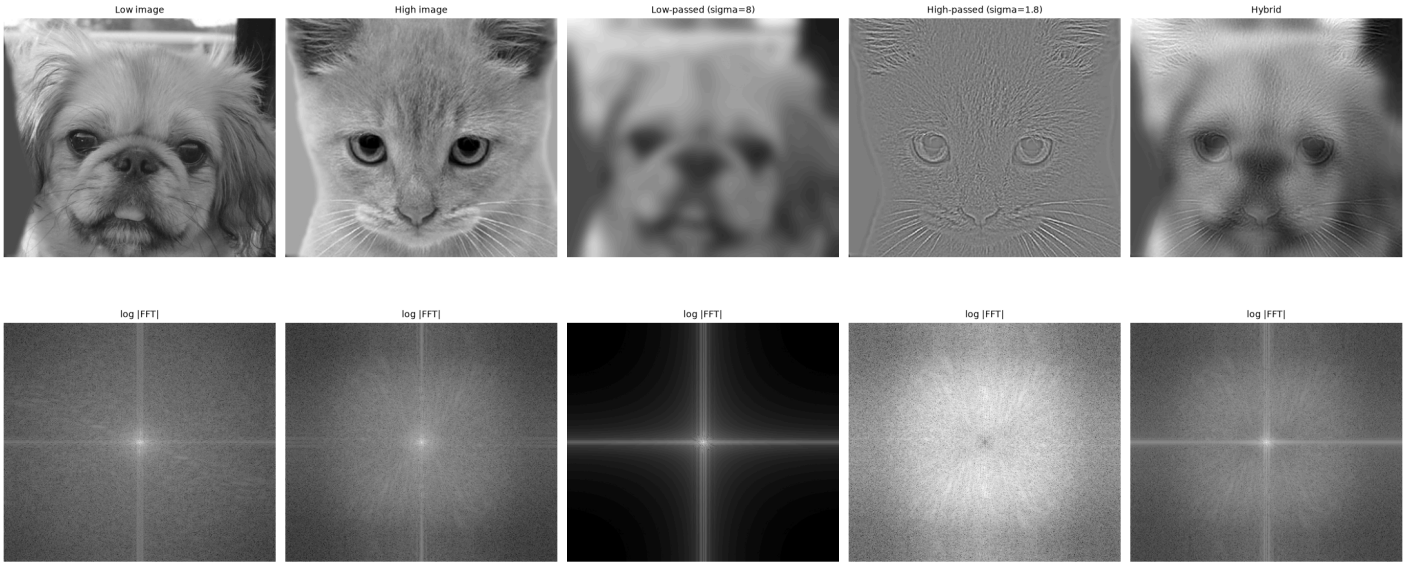
Full process for one hybrid!

For my favourite result I used the dog (low frequencies) and the kitten (high frequencies). The dog's face is made of large soft shapes (the dark eyes, the flat muzzle, the floppy ears), while the kitten has fine fur and whiskers, which is exactly the kind of detail that lives in the high frequencies. Here is the whole pipeline. The cat and dog are mostly aligned so I left them as is, but otherwise I'd first select the eyes similar to Derek and Nutmeg. Because these photos are small, I also made the aligned canvas the same size as the photos instead of the default portrait canvas, otherwise the empty border gets filled with mirrored copies of the image.

Images of a cat and dog

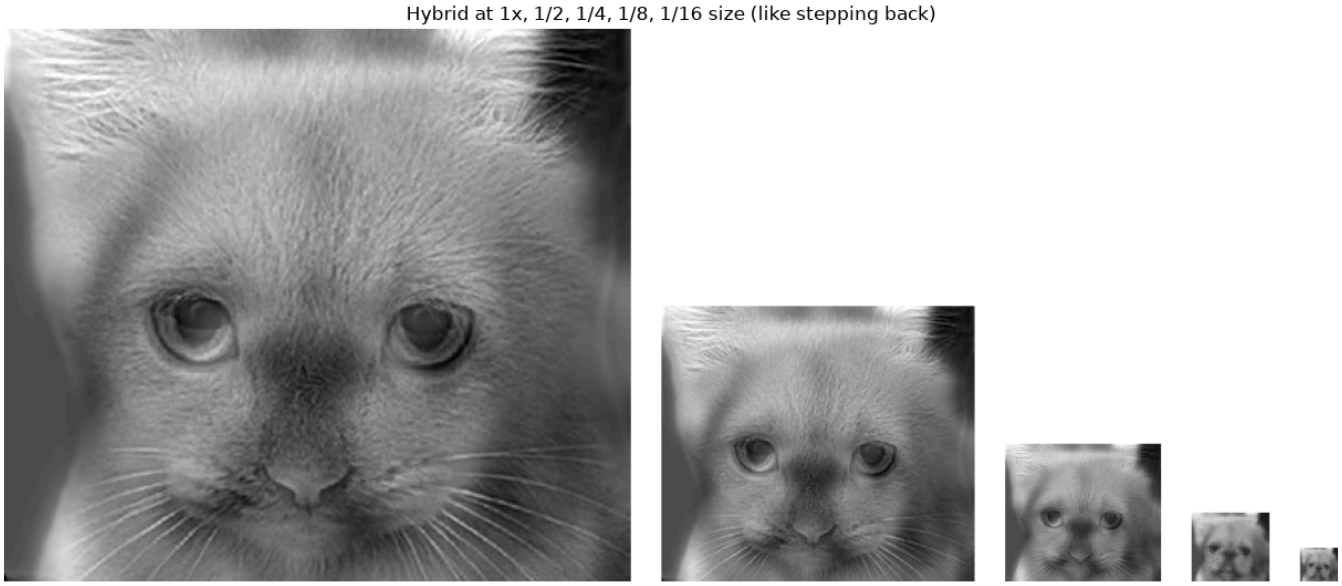
Next is the frequency analysis. The top row is the two aligned grayscale inputs, the low-passed dog, the high-passed cat (shown around mid-gray), and the hybrid; the bottom row is the log magnitude of the Fourier transform of each. The low-passed spectrum is bright only near the centre, because the Gaussian removed everything away from the origin, while

the high-passed spectrum is the opposite: the centre is dark and the outer values remain. The hybrid's spectrum has both, which is why it can look like either animal. (The bright horizontal and vertical lines through the middle of every spectrum come from the image borders, where the picture is cut off abruptly, and are not part of the content.)



Aligned inputs, filtered images, and hybrid (top) with their log Fourier magnitudes (bottom), σ_{low} = 8.0, σ_{high} = 1.8

Finally, the result at different viewing distances. Since I can't move you away from your screen, the same hybrid is shown at full size, then 1/2, 1/4, 1/8 and 1/16 of the size: up close it is the kitten (the eyes, whiskers and fur), and as it shrinks the kitten's fine detail disappears and the dog takes over (by half size the dog already reads clearly).



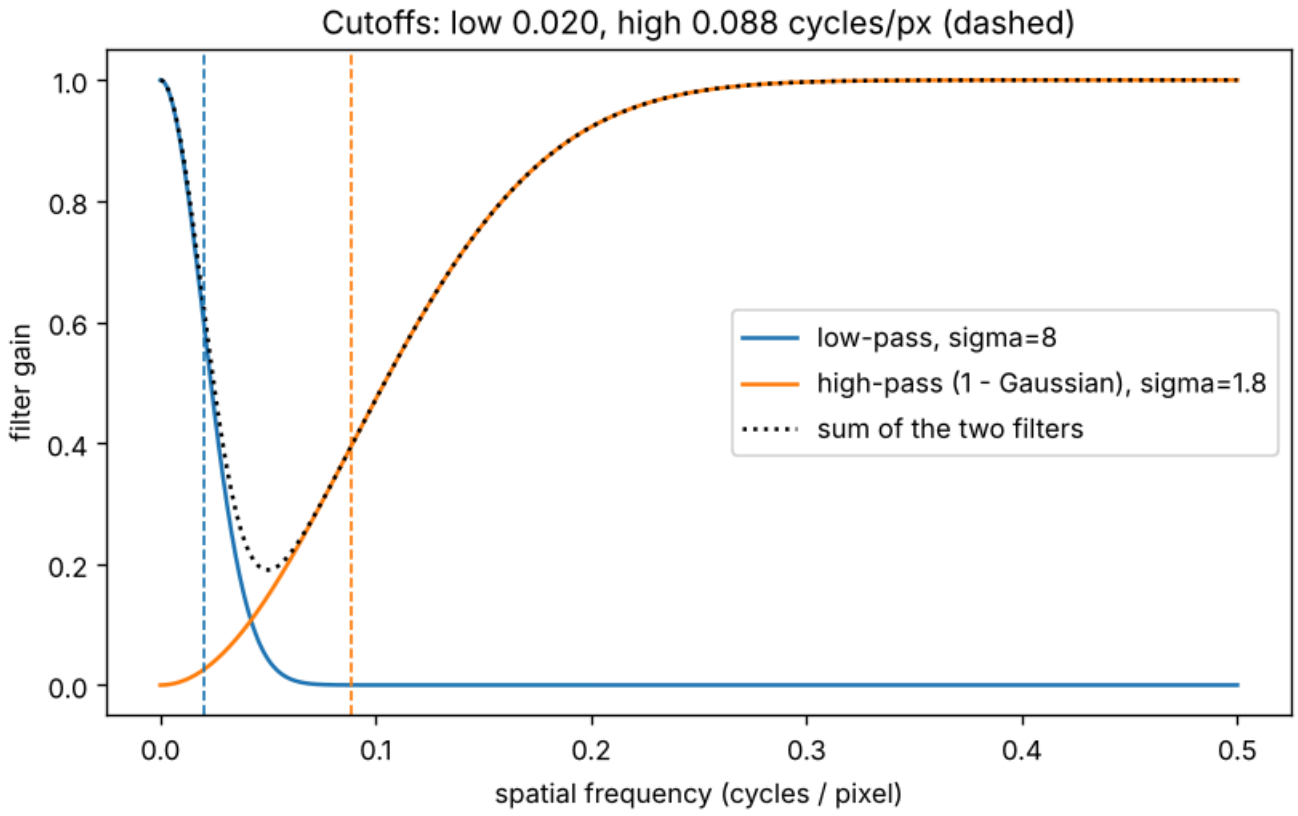
Hybrid image of a cat and dog at different distances, σ_{low} = 8.0, σ_{high} = 1.8

The low-pass and high-pass filters, as well as the summation, can also be seen in the frequency domain (bottom row of the frequency-analysis figure above). The low-passed dog keeps only the central values of k-space (low frequencies), so its spectrum is bright near the centre and black further out. The high-passed cat is the opposite: its centre is dark and the outer values (high frequencies) remain. The hybrid on the right has both: the centre of the dog's spectrum plus the

outside of the cat's. Taking the log of the magnitude allows the frequencies to be visualized better; otherwise the low-pass would show as a very thin bright dot.

Choosing the cutoff frequencies. A Gaussian with standard deviation σ pixels has frequency response $\exp(-2\pi^2\sigma^2 f^2)$, so I define its cutoff as $f_c = 1/(2\pi\sigma)$ cycles per pixel (where the gain has fallen to $e^{-1/2} \approx 0.61$). For the dog and cat, $\sigma_{low} = 8$ gives a low-pass cutoff of 0.020 cycles/pixel (features larger than about 50 pixels survive) and $\sigma_{high} = 1.8$ gives a high-pass cutoff of 0.088 cycles/pixel (features smaller than about 11 pixels are kept). The high-pass cutoff is more than four times the low-pass cutoff, which leaves a gap of mid frequencies that neither image contributes much to: the dotted "sum of filters" curve in the plot below dips to about 0.19 near 0.05 cycles/pixel. This choice means the dog supplies only the coarse shapes, while the cat supplies only its finest detail (whiskers, fur and the outline of the eyes), so the two identities hardly overlap in the same frequency band. The cutoffs themselves were chosen by experimenting with the viewing distance, as with the two σ pairs for Derek and Nutmeg above. The same formula gives the cutoffs for the Derek and Nutmeg pairs:

Hybrid	σ_{low}	σ_{high}	Low-pass cutoff (cycles/pixel)	High-pass cutoff (cycles/pixel)
Dog + cat	8.0	1.8	0.020	0.088
Derek + Nutmeg	7.0	3.5	0.023	0.045
Derek + Nutmeg	10.0	1.5	0.016	0.106

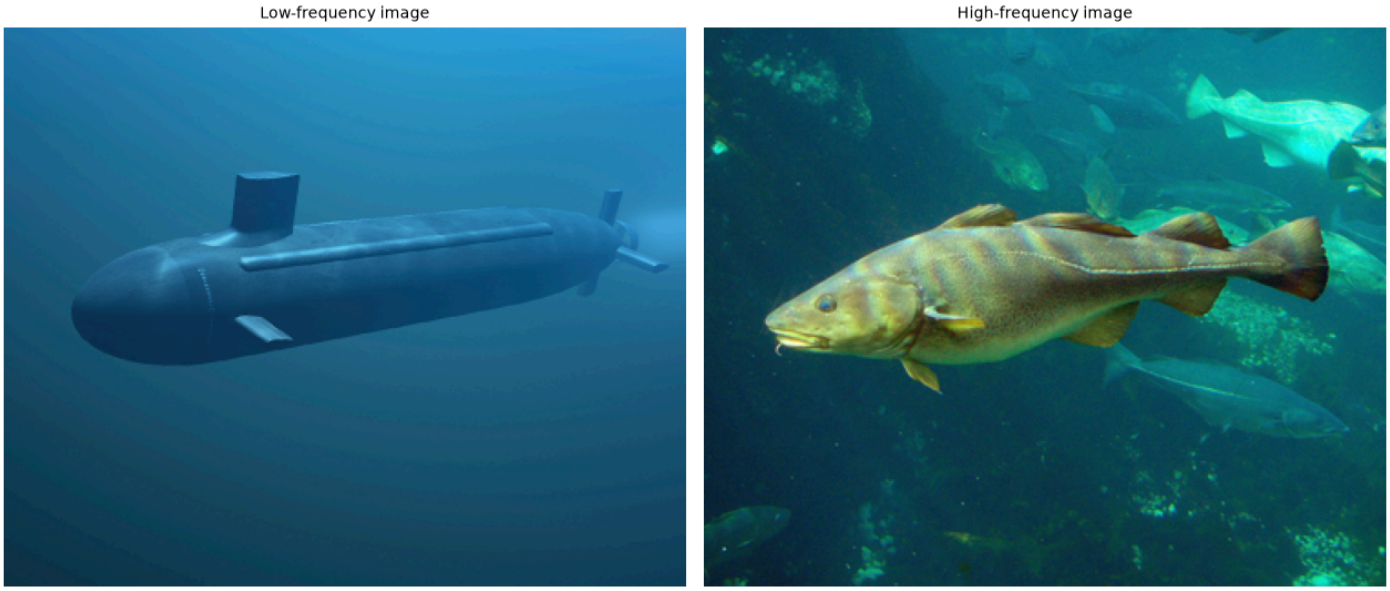


Frequency response of the low-pass (σ = 8) and high-pass (σ = 1.8) filters, with their cutoffs (dashed) at 0.020 and 0.088 cycles/pixel and the sum of the two filters (dotted).

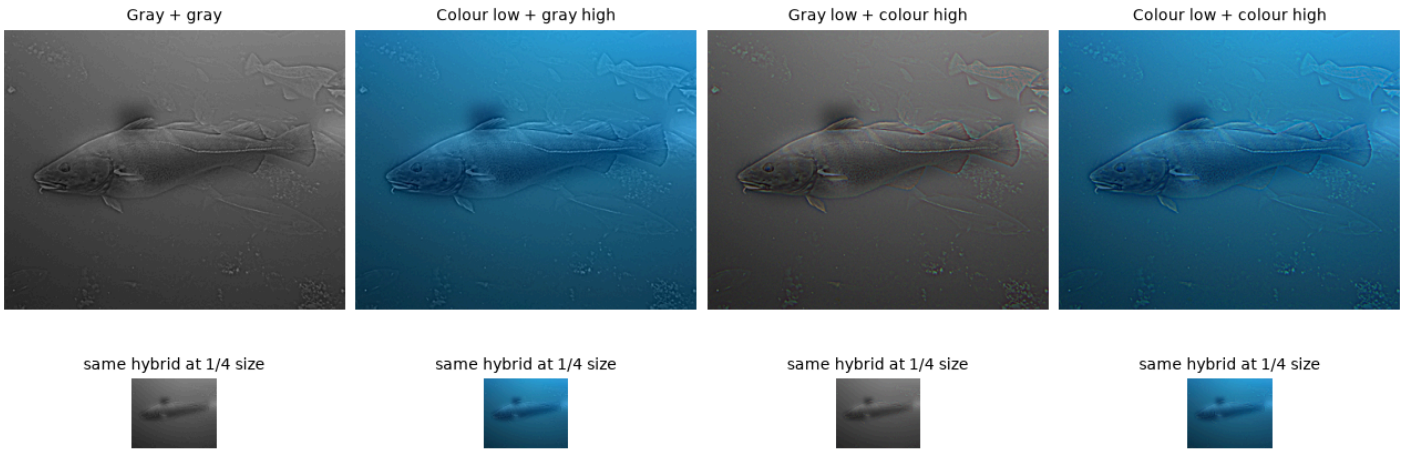
Bells & Whistles

Now that we've got some good hybrid images in black and white, we can explore getting the colour back in these images. So I tried seeing whether the colour from the low-frequency image or the high-frequency image works better!

So I've tried a few combinations below with submarine and fish.



Images of submarine and fish



Colour images of submarine and fish: gray + gray, colour low + gray high, gray low + colour high, colour + colour

Looking at the different combinations, we can see that the colour from the low-frequency image tends to produce more visually appealing results, and it matches that of the hybrid image of colour components from both the low-frequency and high-frequency images. The reason is that colour information is mostly low-frequency (large smooth regions of hue), so the low-pass image carries almost all of it anyway, while the high-pass image is just a thin layer of edges whose

colour is barely visible, and when it is visible it can fight the low-frequency colour. Colour on the high-frequency image alone does little to help the far-away interpretation, since from a distance we only see the low-frequency image's (gray) colour.

MultiResolution Blending and the Oraple Journey

2.3 Gaussian and Laplacian Stacks

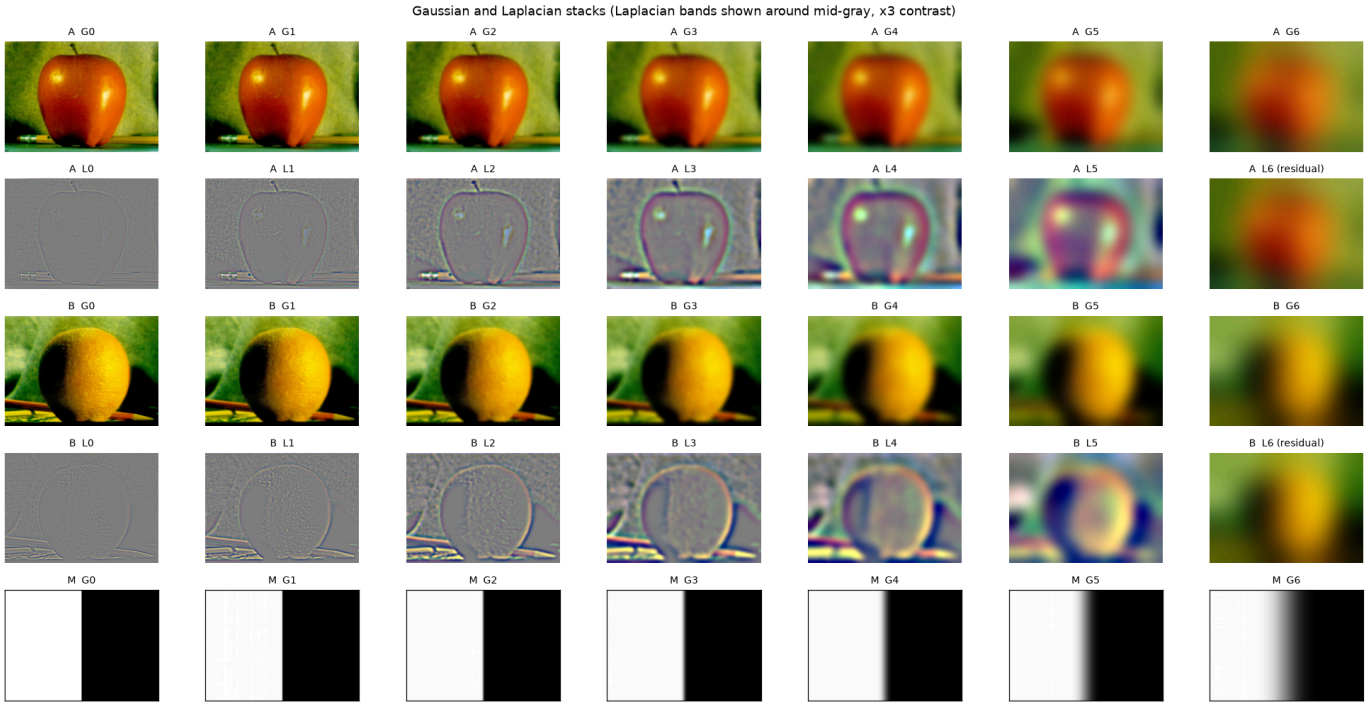
Another way to combine images is to blend or stitch them together, kind of how our phones take panorama photos. Panoramas are created when our phones take successive images of the scene, then are stitched together. There might be cases where we might want to do this manually, as panoramas are low resolution, so if you take several images of a scene using a fancy camera, then you can make a high-resolution panorama. Both manual and phone panoramas can have issues with the seams between the images, so we can use multi-resolution blending to help with this.

A technique we can use is multi-resolution blending, which involves creating Gaussian and Laplacian pyramids of the images and then combining them at different scales. This can also be accomplished with Gaussian and Laplacian stacks, which are the same as pyramids, but without the downsampling, so every level keeps the full image size. Our starting materials are the two images that are being combined, and a mask. To start, we'll use a mask where the left half of the product is of image A and the right half is of image B.

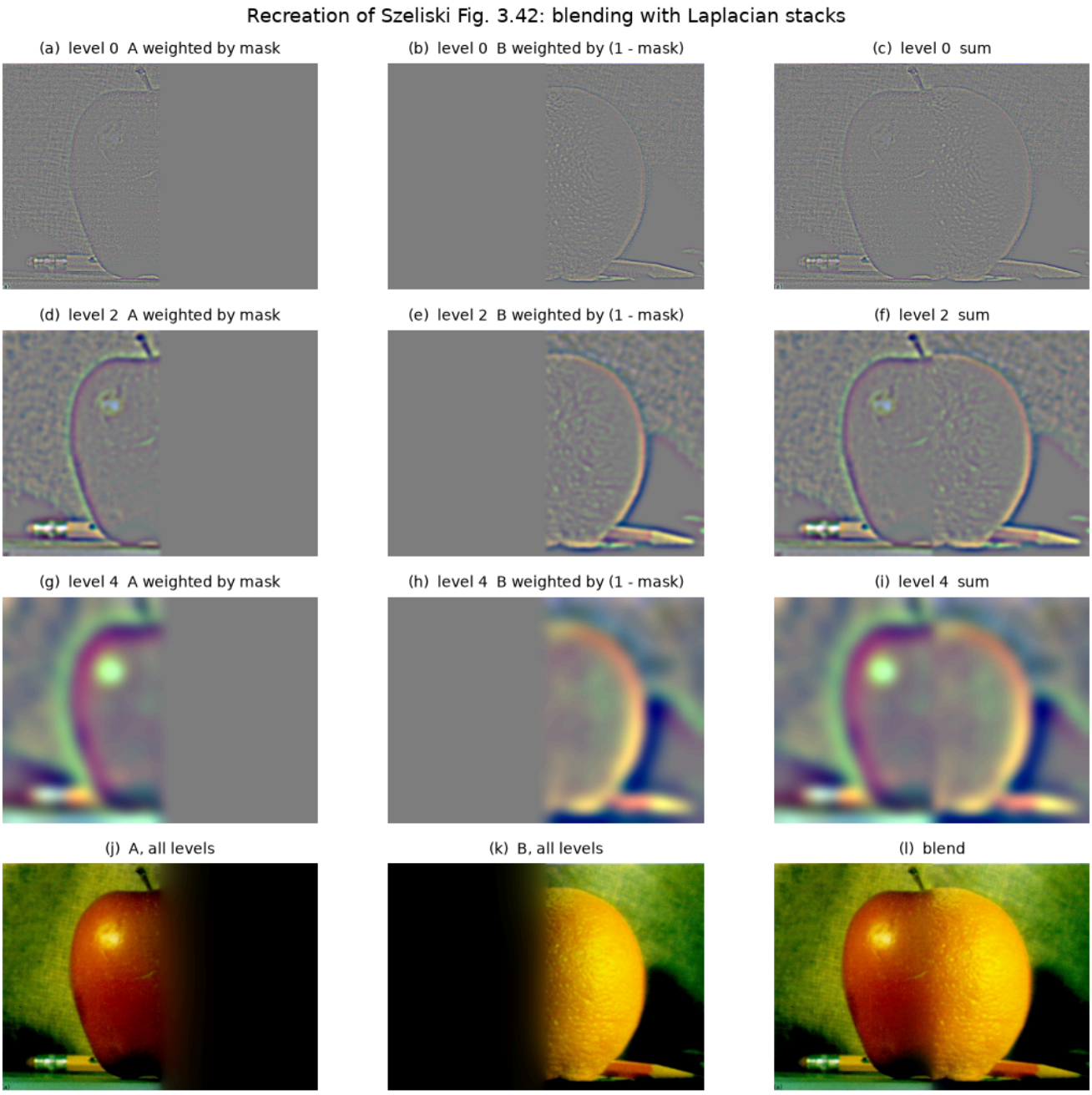
How the stacks are built. The Gaussian stack starts with the original image as level 0 and then blurs it with a Gaussian whose σ doubles at each level (I used 5 levels with σ = 2, 4, 8, 16, 32 pixels), mimicking the blur a downsampled pyramid would have. The Laplacian stack is the difference between neighbouring Gaussian levels, $\mathbf{L_l = G_l - G_{l+1}}$, so each level holds one band of frequencies (the first level the finest detail, the later ones coarser). The last entry is just the final Gaussian level, the low-frequency residual, so that adding all the Laplacian levels back together gives the original image exactly (I check this: the error is about 1e-7). We then construct Gaussian and Laplacian stacks for images A and B at these different **σ** values, and also build a **Gaussian stack of the mask**. The blend is formed level by level, **blend =** $\mathbf{\sum_l L_l(A) \cdot G_l(mask) + L_l(B) \cdot (1 - G_l(mask))}$. Because the mask gets blurrier at the coarse levels, the transition between the two images is soft for low frequencies and sharp for fine detail, which is what hides the seam.

▼ Show Gaussian/Laplacian stack implementation

```
def gaussian_stack(img, levels, sigma):
    """[G_0 .. G_levels]: full-size images, blurrier every level (no downsampling)."""
    return [img] + [blur(img, sigma * 2 ** (l - 1)) for l in range(1, levels + 1)]

def laplacian_stack(g):
    """[L_0 .. L_levels]: bands, plus the low-frequency remainder as the last entry."""
    return [g[l] - g[l + 1] for l in range(len(g) - 1)] + [g[-1]]

def blend_stacks(lap_a, lap_b, mask_stack):
    """Per-level blend; returns the weighted bands of A, of B, and their sum."""
    wa = [la * m for la, m in zip(lap_a, mask_stack)]
    wb = [lb * (1 - m) for lb, m in zip(lap_b, mask_stack)]
    return wa, wb, [a + b for a, b in zip(wa, wb)]

ga, gb, gm = (gaussian_stack(x, 5, 2.0) for x in (a, b, mask3))   # stacks of A, B AND the mask
la, lb = laplacian_stack(ga), laplacian_stack(gb)
wa, wb, wsum = blend_stacks(la, lb, gm)
blended = np.clip(sum(wsum), 0, 1)
```



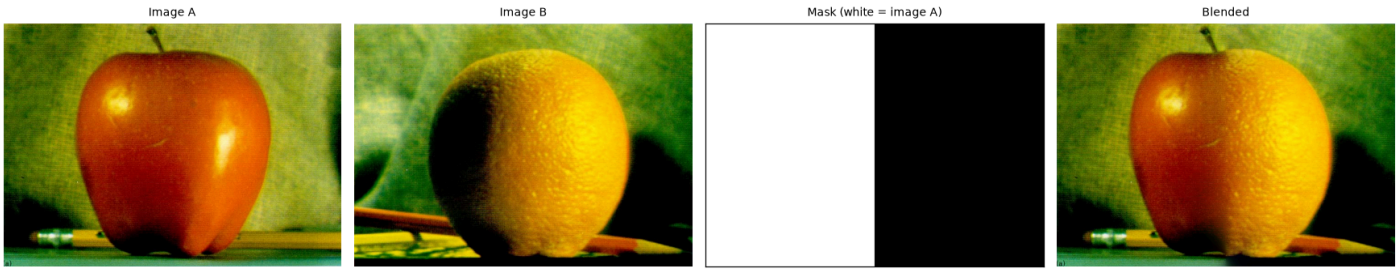
Gaussian and Laplacian stacks of the apple (A) and orange (B), and the Gaussian stack of the mask. Laplacian bands are shown around mid-gray at 3x contrast; the last Laplacian entry is the low-frequency residual.



Laplacian stacks blending details: recreation of Figure 3.42 (a)–(l). Rows 1–3 are the high, medium and low frequency levels (Laplacian levels 0, 2, 4) of the apple weighted by the mask, the orange weighted by (1 − mask), and their sum; the bottom row shows all levels added together.

2.4 Multi-resolution Blending

Putting the stacks to use: here is the classic oraple, made with a vertical step mask (apple on the left half, orange on the right). The mask's Gaussian stack softens the seam more at the coarse levels than at the fine ones, so the colours and shading blend gradually but the apple and orange textures stay sharp up to the seam. All blending is done on the colour (RGB) images, with each channel processed separately, which keeps the colour of both fruits intact.



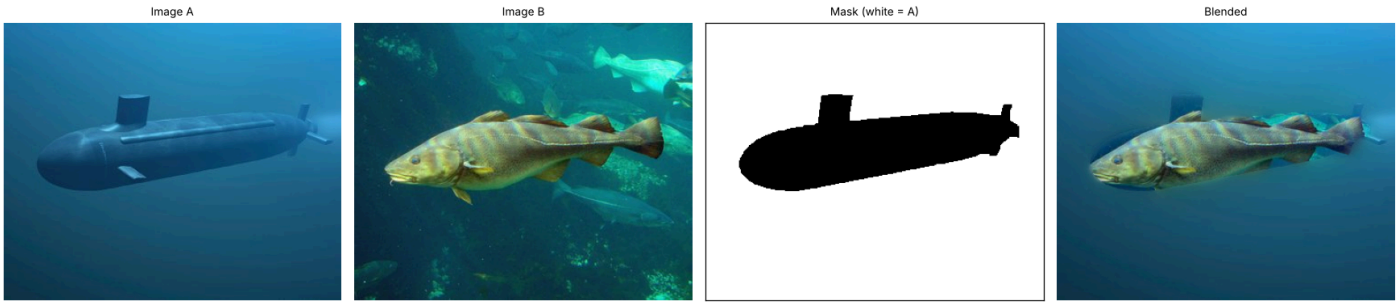
Multi-resolution blending of an apple and orange

An irregular mask: the "fishmarine"

For an irregular mask, I wanted a mask in the shape of an object. I cut out the silhouette of the submarine with OpenCV's GrabCut (given a rectangle around the object), cleaned it up by closing small holes and keeping the largest blob, and inverted it, so that the mask is white (take image A, the submarine photo) everywhere *except* inside the submarine's shape. Image B is a photo of a fish, so the result is a submarine-shaped window filled with fish skin, floating in the submarine's own water. Both photos are underwater scenes in similar colours, so the low-frequency bands blend well.

▼ **Show GrabCut mask generation code**

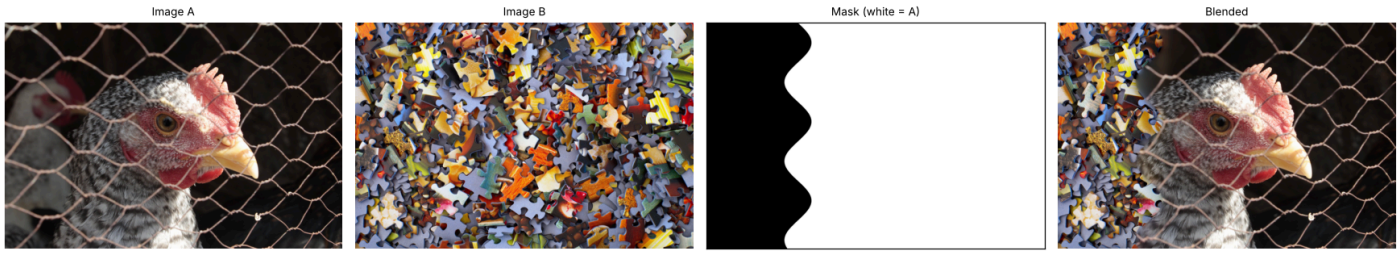
```
def grabcut(img_bgr, rect, iters=8):
    h, w = img_bgr.shape[:2]
    x, y, rw, rh = rect
    r = (int(x * w), int(y * h), int(rw * w), int(rh * h))
    mask = np.zeros((h, w), np.uint8)
    bgd, fgd = np.zeros((1, 65)), np.zeros((1, 65))
    cv2.grabCut(img_bgr, mask, r, bgd, fgd, iters, cv2.GC_INIT_WITH_RECT)
    m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255, 0).astype(np.uint8)
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, np.ones((9, 9), np.uint8))      # fill small holes
    n, labels, stats, _ = cv2.connectedComponentsWithStats(m)
    return ((labels == 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])) * 255).astype(np.uint8)  # largest blob

# python make_mask.py grabcut submarine.jpg mask_sub.png --rect 0.08 0.25 0.85 0.5 --invert
# python simple_2_3_stacks.py submarine.jpg fish.jpg --name fishmarine --mask mask_sub.png
```



Submarine (A), fish (B), the irregular GrabCut mask (white = A), and the blended image

Another idea: the puzzle chicken

For another blend I wanted a seam that looks like the edge of the wired fence, so I used a wavy (sinusoidal) seam as the mask. The seam sits to the left of the chicken's face so the face isn't cut, and the left side, where the blurry background hen was, is filled with a heap of puzzle pieces. These images are large, so I downscaled them to 900 pixels wide and used a larger first-level σ of 3 (3, 6, 12, 24, 48 pixels) so the transition is still smooth relative to the image size.



Chicken (A), puzzle pieces (B), the wavy mask (white = A), and the blended image

Bells & Whistles

The choices I made and why: (1) **Colour**: every blend is done per RGB channel instead of in grayscale, which keeps the colours of both inputs intact and avoids the gray-looking seam you get when only luminance is blended. (2) **Blur doubling per level**: σ doubling imitates the way a pyramid halves the resolution, so the five bands cover roughly the whole frequency range from fine texture to overall shading. (3) **Blurring the mask by the same amount as the bands**: this is what makes the seam wide for coarse bands and narrow for fine ones. Using a hard mask at all levels gives a visible seam (the stack version of the algorithm on page 226 of Szeliski fails for exactly this reason), and using one blurry mask at all levels makes the fine detail ghost across the seam. (4) **Number of levels and σ scale with image size**: for a 375×307 image σ starts at 2, and for the 900-pixel chicken at 3, since the same σ is a much narrower transition in a bigger image. (5) **Mask shape**: GrabCut is useful when the seam should follow an object, but the mask only needs to be roughly correct, because the blurred mask hides small inaccuracies along the object's boundary.

What I Learned

The most important thing I learned from this project is that there is a lot that we can do with images related to frequency. We can sharpen images, create hybrid images, and blend images together. The most interesting part of this project was the hybrid images, where we can combine two images together to create a new image that looks different depending on the viewing distance. This is a very interesting concept that I had never thought about before, and it was fun to experiment with different images and see how they looked at different distances.